```
CREATE DATABASE banking_db;

USE banking_db;

CREATE TABLE bank_accounts (
   account_number BIGINT PRIMARY KEY,
   customer_name VARCHAR(100) NOT NULL,
   password VARCHAR(255) NOT NULL,
   email VARCHAR(100) NOT NULL,
   phone VARCHAR(15) NOT NULL,
   date_of_birth DATE NOT NULL,
   gender VARCHAR(10) NOT NULL,
   address VARCHAR(200) NOT NULL,
   account_type VARCHAR(20) NOT NULL,
   balance DECIMAL(12 , 2 ) NOT NULL
);

INSERT INTO bank_accounts
(account_number, customer_name, password, email, phone,
 date_of_birth, gender, address, account_type, balance)
VALUES
(100000000001, 'Aarav Sharma', 'Aarav@123', 'aarav@example.com', '9000000001',
'1995-04-12', 'Male', 'Kolkata, West Bengal', 'Savings', 25000.00),
(100000000002, 'Priya Das', 'Priya@123', 'priya@example.com', '9000000002', '1998-07-
23', 'Female', 'Howrah, West Bengal', 'Savings', 32000.50),
```

(100000000003, 'Rahul Verma', 'Rahul@123', 'rahul@example.com', '9000000003', '1992-01-15', 'Male', 'Delhi, India', 'Current', 85000.00),

(100000000004, 'Sneha Roy', 'Sneha@123', 'sneha@example.com', '9000000004', '2000-11-09', 'Female', 'Durgapur, West Bengal', 'Savings', 18000.00),

(100000000005, 'Arjun Singh', 'Arjun@123', 'arjun@example.com', '9000000005', '1994-06-18', 'Male', 'Mumbai, Maharashtra', 'Salary', 54000.00),

(100000000006, 'Ananya Bose', 'Ananya@123', 'ananya@example.com', '9000000006', '1997-02-25', 'Female', 'Siliguri, West Bengal', 'Savings', 41000.75),

(100000000007, 'Rohan Gupta', 'Rohan@123', 'rohan@example.com', '9000000007', '1991-09-14', 'Male', 'Pune, Maharashtra', 'Current', 125000.00),

(100000000008, 'Ishita Sen', 'Ishita@123', 'ishita@example.com', '9000000008', '1999-03-30', 'Female', 'Bhubaneswar, Odisha', 'Savings', 27500.00),

(100000000009, 'Vikram Patel', 'Vikram@123', 'vikram@example.com', '9000000009', '1993-12-05', 'Male', 'Ahmedabad, Gujarat', 'Salary', 67000.00),

(100000000010, 'Kavya Nair', 'Kavya@123', 'kavya@example.com', '9000000010', '1996-08-17', 'Female', 'Kochi, Kerala', 'Savings', 36000.00),

(100000000011, 'Aditya Kumar', 'Aditya@123', 'aditya@example.com', '9000000011', '1990-05-22', 'Male', 'Patna, Bihar', 'Current', 92000.00),

(100000000012, 'Meera Iyer', 'Meera@123', 'meera@example.com', '9000000012', '2001-10-11', 'Female', 'Chennai, Tamil Nadu', 'Savings', 15500.00),

(100000000013, 'Siddharth Jain', 'Siddharth@123', 'siddharth@example.com', '9000000013', '1994-07-07', 'Male', 'Jaipur, Rajasthan', 'Salary', 48000.00),

(100000000014, 'Riya Ghosh', 'Riya@123', 'riya@example.com', '9000000014', '1998-01-28', 'Female', 'Kolkata, West Bengal', 'Savings', 29500.00),

(100000000015, 'Karan Malhotra', 'Karan@123', 'karan@example.com', '9000000015', '1992-04-19', 'Male', 'Chandigarh, India', 'Current', 110000.00),

(100000000016, 'Pooja Reddy', 'Pooja@123', 'pooja@example.com', '9000000016', '1997-09-03', 'Female', 'Hyderabad, Telangana', 'Savings', 22500.00),

(100000000017, 'Amit Chatterjee', 'Amit@123', 'amit@example.com', '9000000017', '1989-02-16', 'Male', 'Howrah, West Bengal', 'Salary', 73500.00),

(100000000018, 'Nisha Kapoor', 'Nisha@123', 'nisha@example.com', '9000000018', '2000-06-27', 'Female', 'Lucknow, Uttar Pradesh', 'Savings', 19000.00),

(100000000019, 'Dev Mehta', 'Dev@123', 'dev@example.com', '9000000019', '1995-11-13', 'Male', 'Surat, Gujarat', 'Current', 88000.00),

(100000000020, 'Tanya Dutta', 'Tanya@123', 'tanya@example.com', '9000000020', '1999-12-21', 'Female', 'Kharagpur, West Bengal', 'Savings', 31500.00);

```
USE banking_db;

-- Add a role column so one account can be an admin.
ALTER TABLE bank_accounts
ADD COLUMN role VARCHAR(10) NOT NULL DEFAULT 'CUSTOMER';

-- Designate one existing account as the administrator.
-- Demo admin account: 100000000001
UPDATE bank_accounts
SET role = 'ADMIN'
WHERE account_number = 100000000001;

-- Store deposits, withdrawals and transfers for the passbook.
CREATE TABLE IF NOT EXISTS transactions (
  transaction_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  account_number BIGINT NOT NULL,
  related_account BIGINT NULL,
  transaction_type VARCHAR(20) NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  balance_after DECIMAL(12,2) NOT NULL,
```

```
    description VARCHAR(255) NOT NULL,
    transaction_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_account_time (account_number, transaction_time)
);
```

```
package CaseStudy2;

import java.sql.*;
import java.math.BigDecimal;
import java.util.Scanner;
import java.security.SecureRandom;
import java.security.MessageDigest;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import java.util.Base64;

public class Main {
    private static final String URL =
            "jdbc:mysql://localhost:3306/banking_db";
```

```
private static final String DB_USER = "root";
private static final String DB_PASSWORD = "Pkm@123";

private static final Scanner sc = new Scanner(System.in);
private static long loggedInAccount;
private static boolean admin;

public static void main(String[] args) {
  try (Connection con =
          DriverManager.getConnection(URL, DB_USER, DB_PASSWORD)) {

    System.out.println("===== JAVA BANKING SYSTEM =====");

    boolean running = true;
    while (running) {
      System.out.println("\n1. Login");
      System.out.println("0. Exit");

      switch (readInt("Choose: ")) {
        case 1 -> {
          if (login(con)) {
            if (admin) adminMenu(con);
            else customerMenu(con);
          }
        }
        case 0 -> running = false;
        default -> System.out.println("Invalid choice.");
      }
```

```
        }
    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }

    System.out.println("Thank you for using the bank.");
}

// ---------------- LOGIN ----------------

private static boolean login(Connection con) throws SQLException {
    long account = readLong("Account number: ");
    String password = readLine("Password: ");

    String sql = "SELECT password, role FROM bank_accounts " +
            "WHERE account_number = ?";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setLong(1, account);

        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) {
                System.out.println("Account not found.");
                return false;
            }

            String stored = rs.getString("password");
```

```
            if (!verifyPassword(password, stored)) {
                System.out.println("Incorrect password.");
                return false;
            }

            // Upgrade old demo plaintext passwords on successful login.
            if (!stored.startsWith("PBKDF2$")) {
                try (PreparedStatement update = con.prepareStatement(
                        "UPDATE bank_accounts SET password=? " +
                                "WHERE account_number=?")) {
                    update.setString(1, hashPassword(password));
                    update.setLong(2, account);
                    update.executeUpdate();
                }
            }

            loggedInAccount = account;
            admin = "ADMIN".equals(rs.getString("role"));

            System.out.println("Login successful.");
            System.out.println(admin ? "Administrator access."
                    : "Customer access.");
            return true;
        }
    }
}

// ---------- CUSTOMER MENU ----------
```

```
private static void customerMenu(Connection con) throws SQLException {
  boolean active = true;

  while (active) {
    System.out.println("\n===== CUSTOMER MENU =====");
    System.out.println("1. Account details");
    System.out.println("2. Check balance");
    System.out.println("3. Deposit");
    System.out.println("4. Withdraw");
    System.out.println("5. Transfer money");
    System.out.println("6. View passbook");
    System.out.println("7. Change password");
    System.out.println("0. Logout");

    switch (readInt("Choose: ")) {
      case 1 -> accountDetails(con, loggedInAccount);
      case 2 -> checkBalance(con, loggedInAccount);
      case 3 -> deposit(con);
      case 4 -> withdraw(con);
      case 5 -> transfer(con);
      case 6 -> passbook(con, loggedInAccount);
      case 7 -> changePassword(con);
      case 0 -> active = false;
      default -> System.out.println("Invalid choice.");
    }
  }
```

```
    admin = false;
    loggedInAccount = 0;
    System.out.println("Logged out.");
}

// ---------------- ADMIN MENU ----------------

private static void adminMenu(Connection con) throws SQLException {
    boolean active = true;

    while (active) {
        System.out.println("\n===== ADMIN MENU =====");
        System.out.println("1. List all accounts");
        System.out.println("2. Search account");
        System.out.println("3. Create account");
        System.out.println("4. Update customer record");
        System.out.println("5. Delete customer account");
        System.out.println("6. View account passbook");
        System.out.println("7. Change own password");
        System.out.println("0. Logout");

        switch (readInt("Choose: ")) {
            case 1 -> listAccounts(con);
            case 2 -> {
                long id = readLong("Account number: ");
                accountDetails(con, id);
            }
            case 3 -> createAccount(con);
```

```
            case 4 -> updateAccount(con);
            case 5 -> deleteAccount(con);
            case 6 -> {
                long id = readLong("Account number: ");
                passbook(con, id);
            }
            case 7 -> changePassword(con);
            case 0 -> active = false;
            default -> System.out.println("Invalid choice.");
        }
    }

    admin = false;
    loggedInAccount = 0;
    System.out.println("Logged out.");
}

// ---------------- ACCOUNT DETAILS ----------------

private static void accountDetails(Connection con, long id)
        throws SQLException {
    String sql = "SELECT account_number, customer_name, email, phone, " +
            "date_of_birth, gender, address, account_type, balance, role " +
            "FROM bank_accounts WHERE account_number=?";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setLong(1, id);
```

```
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) {
                System.out.println("Account not found.");
                return;
            }

            System.out.println("Account: " + rs.getLong("account_number"));
            System.out.println("Name: " + rs.getString("customer_name"));
            System.out.println("Email: " + rs.getString("email"));
            System.out.println("Phone: " + rs.getString("phone"));
            System.out.println("DOB: " + rs.getDate("date_of_birth"));
            System.out.println("Gender: " + rs.getString("gender"));
            System.out.println("Address: " + rs.getString("address"));
            System.out.println("Type: " + rs.getString("account_type"));
            System.out.println("Balance: Rs. " + rs.getBigDecimal("balance"));

            if (admin) {
                System.out.println("Role: " + rs.getString("role"));
            }
        }
    }
}

private static void checkBalance(Connection con, long id)
        throws SQLException {
    try (PreparedStatement ps = con.prepareStatement(
            "SELECT balance FROM bank_accounts WHERE account_number=?")) {
        ps.setLong(1, id);
```

```
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                System.out.println("Available balance: Rs. " +
                        rs.getBigDecimal("balance"));
            }
        }
    }
}

private static void listAccounts(Connection con) throws SQLException {
    String sql = "SELECT account_number, customer_name, email, " +
            "account_type, balance, role FROM bank_accounts " +
            "ORDER BY account_number";

    try (PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {

        System.out.printf("%-15s %-22s %-12s %-12s %-10s%n",
                "Account", "Name", "Type", "Balance", "Role");

        while (rs.next()) {
            System.out.printf("%-15d %-22s %-12s %-12s %-10s%n",
                    rs.getLong("account_number"),
                    rs.getString("customer_name"),
                    rs.getString("account_type"),
                    rs.getBigDecimal("balance"),
                    rs.getString("role"));
```

```
        }
    }
}

// ---------------- DEPOSIT ----------------

private static void deposit(Connection con) throws SQLException {
    BigDecimal amount = readAmount("Deposit amount: ");

    con.setAutoCommit(false);
    try {
        BigDecimal balance = changeBalance(
                con, loggedInAccount, amount);

        recordTransaction(con, loggedInAccount, null,
                "DEPOSIT", amount, balance, "Cash deposit");

        con.commit();
        System.out.println("Deposit successful. Balance: Rs. " + balance);
    } catch (SQLException | RuntimeException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(true);
    }
}

// ---------------- WITHDRAW ----------------
```

```
private static void withdraw(Connection con) throws SQLException {
    BigDecimal amount = readAmount("Withdrawal amount: ");

    con.setAutoCommit(false);
    try {
        BigDecimal balance = changeBalance(
                con, loggedInAccount, amount.negate());

        recordTransaction(con, loggedInAccount, null,
                "WITHDRAWAL", amount, balance, "Cash withdrawal");

        con.commit();
        System.out.println("Withdrawal successful. Balance: Rs. " + balance);
    } catch (SQLException | RuntimeException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(true);
    }
}

// ---------------- TRANSFER ----------------

private static void transfer(Connection con) throws SQLException {
    long recipient = readLong("Recipient account number: ");

    if (recipient == loggedInAccount) {
```

```
    System.out.println("You cannot transfer to your own account.");
    return;
}

BigDecimal amount = readAmount("Transfer amount: ");

con.setAutoCommit(false);
try {
    BigDecimal senderBalance = changeBalance(
            con, loggedInAccount, amount.negate());

    BigDecimal recipientBalance = changeBalance(
            con, recipient, amount);

    recordTransaction(con, loggedInAccount, recipient,
            "TRANSFER_OUT", amount, senderBalance,
            "Transfer to " + recipient);

    recordTransaction(con, recipient, loggedInAccount,
            "TRANSFER_IN", amount, recipientBalance,
            "Transfer from " + loggedInAccount);

    con.commit();
    System.out.println("Transfer successful.");
} catch (SQLException | RuntimeException e) {
    con.rollback();
    throw e;
} finally {
```

```
      con.setAutoCommit(true);
    }
  }

  // Update balance atomically; reject missing accounts and overdrafts.
  private static BigDecimal changeBalance(
      Connection con, long account, BigDecimal delta)
      throws SQLException {

    String sql = "UPDATE bank_accounts SET balance=balance+? " +
        "WHERE account_number=? AND balance+?>=0";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
      ps.setBigDecimal(1, delta);
      ps.setLong(2, account);
      ps.setBigDecimal(3, delta);

      if (ps.executeUpdate() != 1) {
        throw new SQLException(
            "Account not found or insufficient balance: " + account);
      }
    }

    try (PreparedStatement ps = con.prepareStatement(
        "SELECT balance FROM bank_accounts WHERE account_number=?")) {
      ps.setLong(1, account);

      try (ResultSet rs = ps.executeQuery()) {
```

```
            if (!rs.next()) throw new SQLException("Account not found.");
            return rs.getBigDecimal("balance");
        }
    }
}

private static void recordTransaction(
        Connection con, long account, Long related,
        String type, BigDecimal amount, BigDecimal balance,
        String description) throws SQLException {

    String sql = "INSERT INTO transactions " +
            "(account_number, related_account, transaction_type, " +
            "amount, balance_after, description) VALUES (?, ?, ?, ?, ?, ?)";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setLong(1, account);

        if (related == null) ps.setNull(2, Types.BIGINT);
        else ps.setLong(2, related);

        ps.setString(3, type);
        ps.setBigDecimal(4, amount);
        ps.setBigDecimal(5, balance);
        ps.setString(6, description);
        ps.executeUpdate();
    }
}
```

```
// ---------------- PASSBOOK ----------------

private static void passbook(Connection con, long account)
    throws SQLException {

  // Customers may only view their own passbook.
  if (!admin && account != loggedInAccount) {
    System.out.println("Access denied.");
    return;
  }

  String sql = "SELECT transaction_id, transaction_time, " +
      "transaction_type, related_account, amount, balance_after, " +
      "description FROM transactions WHERE account_number=? " +
      "ORDER BY transaction_time, transaction_id";

  try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setLong(1, account);

    try (ResultSet rs = ps.executeQuery()) {
      System.out.println("\n===== PASSBOOK =====");
      System.out.println("ID | Date | Type | Related account | Amount | Balance |
Details");

      boolean found = false;
      while (rs.next()) {
        found = true;
```

```
            System.out.printf("%d | %s | %s | Rs. %s | Rs. %s | %s%n",
                    rs.getLong("transaction_id"),
                    rs.getTimestamp("transaction_time"),
                    rs.getString("transaction_type"),
                    rs.getObject("related_account"),
                    rs.getBigDecimal("amount"),
                    rs.getBigDecimal("balance_after"),
                    rs.getString("description"));
            }

            if (!found) System.out.println("No transactions recorded yet.");
        }
    }
}

// ---------- CHANGE PASSWORD ----------

private static void changePassword(Connection con) throws SQLException {
    String current = readLine("Current password: ");

    try (PreparedStatement ps = con.prepareStatement(
            "SELECT password FROM bank_accounts WHERE account_number=?")) {
        ps.setLong(1, loggedInAccount);

        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next() || !verifyPassword(current, rs.getString("password"))) {
                System.out.println("Current password is incorrect.");
                return;
```

```
        }
      }
    }

    String next = readLine("New password (at least 8 characters): ");
    if (next.length() < 8) {
      System.out.println("Password must be at least 8 characters.");
      return;
    }

    if (next.equals(current)) {
      System.out.println("Choose a different password.");
      return;
    }

    try (PreparedStatement ps = con.prepareStatement(
        "UPDATE bank_accounts SET password=? WHERE account_number=?")) {
      ps.setString(1, hashPassword(next));
      ps.setLong(2, loggedInAccount);
      ps.executeUpdate();
    }

    System.out.println("Password changed successfully.");
}

// ---------------- ADMIN: CREATE ACCOUNT ----------------

private static void createAccount(Connection con) throws SQLException {
```

```
long id = readLong("New account number: ");
String name = readLine("Customer name: ");
String pass = readLine("Initial password (at least 8 chars): ");

if (pass.length() < 8) {
   System.out.println("Password must be at least 8 characters.");
   return;
}

String email = readLine("Email: ");
String phone = readLine("Phone: ");
Date dob = Date.valueOf(readLine("DOB (YYYY-MM-DD): "));
String gender = readLine("Gender: ");
String address = readLine("Address: ");
String type = readLine("Account type (Savings/Current/Salary): ");
BigDecimal balance = readAmount("Opening balance: ");

if (balance.signum() < 0) {
   System.out.println("Opening balance cannot be negative.");
   return;
}

String sql = "INSERT INTO bank_accounts " +
      "(account_number, customer_name, password, email, phone, " +
      "date_of_birth, gender, address, account_type, balance, role) " +
      "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 'CUSTOMER')";

try (PreparedStatement ps = con.prepareStatement(sql)) {
```

```
            ps.setLong(1, id);
            ps.setString(2, name);
            ps.setString(3, hashPassword(pass));
            ps.setString(4, email);
            ps.setString(5, phone);
            ps.setDate(6, dob);
            ps.setString(7, gender);
            ps.setString(8, address);
            ps.setString(9, type);
            ps.setBigDecimal(10, balance);
            ps.executeUpdate();
            System.out.println("Customer account created.");
        }
    }

    // ---------- ADMIN: UPDATE ACCOUNT ----------

    private static void updateAccount(Connection con) throws SQLException {
        long id = readLong("Account number to update: ");

        if (id == loggedInAccount) {
            System.out.println("Use Change Password for your own password.");
        }

        System.out.println("1. Name  2. Email  3. Phone  4. Address  5. Account type");
        int choice = readInt("Field: ");

        // Column names are selected from this fixed whitelist.
```

```
    String column;
    switch (choice) {
        case 1 -> column = "customer_name";
        case 2 -> column = "email";
        case 3 -> column = "phone";
        case 4 -> column = "address";
        case 5 -> column = "account_type";
        default -> {
            System.out.println("Invalid field.");
            return;
        }
    }

    String value = readLine("New value: ");
    String sql = "UPDATE bank_accounts SET " + column +
            "=? WHERE account_number=? AND role='CUSTOMER'";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, value);
        ps.setLong(2, id);

        if (ps.executeUpdate() == 1)
            System.out.println("Customer record updated.");
        else
            System.out.println("Customer account not found.");
    }
}
```

```
// ---------------- ADMIN: DELETE ACCOUNT ----------------

private static void deleteAccount(Connection con) throws SQLException {
   long id = readLong("Customer account number to delete: ");

   if (id == loggedInAccount) {
      System.out.println("You cannot delete your own admin account.");
      return;
   }

   String sql = "DELETE FROM bank_accounts " +
         "WHERE account_number=? AND role='CUSTOMER'";

   try (PreparedStatement ps = con.prepareStatement(sql)) {
      ps.setLong(1, id);

      if (ps.executeUpdate() == 1)
         System.out.println("Customer account deleted.");
      else
         System.out.println("Customer account not found.");
   }
}

// ---------------- PASSWORD HASHING ----------------

private static String hashPassword(String password) {
   try {
      int iterations = 120000;
```

```
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);

        PBEKeySpec spec = new PBEKeySpec(
            password.toCharArray(), salt, iterations, 256);

        byte[] hash = SecretKeyFactory.getInstance(
            "PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded();

        spec.clearPassword();

        return "PBKDF2$" + iterations + "$" +
            Base64.getEncoder().encodeToString(salt) + "$" +
            Base64.getEncoder().encodeToString(hash);
    } catch (Exception e) {
        throw new IllegalStateException("Password hashing failed.", e);
    }
}

private static boolean verifyPassword(String input, String stored) {
    if (!stored.startsWith("PBKDF2$")) {
        // Supports the initial fictional sample passwords.
        return MessageDigest.isEqual(
            input.getBytes(java.nio.charset.StandardCharsets.UTF_8),
            stored.getBytes(java.nio.charset.StandardCharsets.UTF_8));
    }

    try {
```

```
            String[] parts = stored.split("\\$");
            int iterations = Integer.parseInt(parts[1]);
            byte[] salt = Base64.getDecoder().decode(parts[2]);
            byte[] expected = Base64.getDecoder().decode(parts[3]);

            PBEKeySpec spec = new PBEKeySpec(
                    input.toCharArray(), salt, iterations, expected.length * 8);

            byte[] actual = SecretKeyFactory.getInstance(
                    "PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded();

            spec.clearPassword();
            return MessageDigest.isEqual(expected, actual);
        } catch (Exception e) {
            return false;
        }
    }

    // ---------- INPUT HELPERS ----------

    private static String readLine(String prompt) {
        System.out.print(prompt);
        return sc.nextLine().trim();
    }

    private static int readInt(String prompt) {
        while (true) {
            try {
```

```
            return Integer.parseInt(readLine(prompt));
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid whole number.");
        }
    }
}

private static long readLong(String prompt) {
    while (true) {
        try {
            return Long.parseLong(readLine(prompt));
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid account number.");
        }
    }
}

private static BigDecimal readAmount(String prompt) {
    while (true) {
        try {
            BigDecimal amount = new BigDecimal(readLine(prompt));

            if (amount.scale() > 2 || amount.signum() <= 0) {
                System.out.println("Enter a positive amount with at most 2 decimal places.");
                continue;
            }

            return amount;
```

```
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid monetary amount.");
        }
      }
   }
}
```